WEB TECHNOLOGIES LAB 6

REGISTRATION NO.:23BCE7226
NAME: K.USHA SRI
SLOT:L37+L38

QUESTION 1: Use MongoDB to implement the following DB operations

1. Create a database called ‘vehicles’ and write a MongoDB query to select database as “vehicles”.

Command:

use vehicles

```
vehicles> use vehicles
already on db vehicles
```

2. Write a MongoDB query to display all the databases.

Command:

Show dbs

```
vehicles> show dbs
admin      40.00 KiB
config    104.00 KiB
local      40.00 KiB
vehicles   80.00 KiB
vehicles>
```

3. Create a collection called ‘two_wheelers’. (use capping) and Create a collection called ‘four_wheelers’.

Command:

db.createCollection("two_wheelers",{capped:true,size:50000})
db.createCollection("four_wheelers")

```
vehicles> db.createCollection("two_wheelers",{capped:true,size:50000})
{ ok: 1 }
vehicles> db.createCollection("four_wheelers")
{ ok: 1 }
```

4. Add 5 two-wheeler details to the collection named ‘two_wheelers’. Each document consists of following fields as bike_name, model (gear or gearless), category (100cc,125cc, 150cc, 200cc), colors_available (red, black, blue, sport red etc) as array,manufacturer, performance (out of 10), timestamp (date and year release) and price.

Command:

```
db.two_wheelers.insertMany([
{bike_name:"Activa",model:"gearless",category:"125cc",colors_available:["red","black"],manufacturer:"Honda",performance:8,timestamp:"2022",price:90000},{bike_name:"Pulsar",model:"gear",category:"150cc",colors_available:["black","blue"],manufacturer:"Bajaj",performance:9,timestamp:"2021",price:120000},{bike_name:"Apache",model:"gear",category:"200cc",colors_available:["red","black"],manufacturer:"TVS",performance:9,timestamp:"2023",price:150000},{bike_name:"FZ",model:"gear",category:"150cc",colors_available:["blue","black"],manufacturer:"Yamaha",performance:8,timestamp:"2022",price:130000},
{bike_name:"Shine",model:"gear",category:"100cc",colors_available:["red","black"],manufacturer:"Honda",performance:7,timestamp:"2020",price:80000}
])
```

```
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b175f04ebccd0ecf7c2911'),
    '1': ObjectId('69b175f04ebccd0ecf7c2912'),
    '2': ObjectId('69b175f04ebccd0ecf7c2913'),
    '3': ObjectId('69b175f04ebccd0ecf7c2914'),
    '4': ObjectId('69b175f04ebccd0ecf7c2915')
  }
}
```

5. Add 5 four-wheeler details to the collection named ‘four_wheelers’. Each document consists of following fields as vehicle_name, model (commercial or own), category (car, lorry, bus, mini truck, heavy truck, containers), variants (vxi, zxi, petrol, diesel etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

Command:

```
db.four_wheelers.insertMany([
{vehicle_name:"Swift",model:"own",category:"car",variants:["vxi","petrol"],manufacturer:"Maruti",performance:8,timestamp:"2021",price:700000},
{vehicle_name:"Innova",model:"own",category:"car",variants:["diesel"],manufacturer:"Toyota",performance:9,timestamp:"2023",price:2000000},
{vehicle_name:"Bolero",model:"own",category:"car",variants:["diesel"],manufacturer:"Mahindra",performance:8,timestamp:"2022",price:1000000},
{vehicle_name:"TataBus",model:"commercial",category:"bus",variants:["diesel"],manufacturer:"Tata",performance:7,timestamp:"2020",price:4000000},
{vehicle_name:"LeylandTruck",model:"commercial",category:"heavy truck",variants:["diesel"],manufacturer:"Ashok Leyland",performance:8,timestamp:"2022",price:3500000}
])
```

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b176024ebccd0ecf7c2916')
    '1': ObjectId('69b176024ebccd0ecf7c2917')
    '2': ObjectId('69b176024ebccd0ecf7c2918')
    '3': ObjectId('69b176024ebccd0ecf7c2919')
    '4': ObjectId('69b176024ebccd0ecf7c291a')
  }
}
```

6. Write a MongoDB query to display all documents available in two_wheelers and four_wheelers.

Command:

```
db.two_wheelers.find()
db.four_wheelers.find()
```

```
vehicles> db.two_wheelers.find()
[
  {
    _id: ObjectId('69b175f04ebccd0ecf7c2911'),
    bike_name: 'Activa',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'red', 'black' ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: '2022',
    price: 90000
  },
  {
    _id: ObjectId('69b175f04ebccd0ecf7c2912'),
    bike_name: 'Pulsar',
    model: 'gear',
    category: '150cc',
    colors_available: [ 'black', 'blue' ],
    manufacturer: 'Bajaj',
    performance: 9,
    timestamp: '2021',
    price: 120000
  },
  {
    _id: ObjectId('69b175f04ebccd0ecf7c2913'),
    bike_name: 'Apache',
    model: 'gear',
    category: '200cc',
    colors_available: [ 'red', 'black' ],
    manufacturer: 'TVS',
    performance: 9,
    timestamp: '2023',
    price: 150000
  },
  {
    _id: ObjectId('69b175f04ebccd0ecf7c2914'),
    bike_name: 'FZ',
    model: 'gear',
    category: '150cc',
    colors_available: [ 'blue', 'black' ],
    manufacturer: 'Yamaha',
    performance: 8,
    timestamp: '2022',
    price: 130000
  },
```

```
  {
    _id: ObjectId('69b175f04ebccd0ecf7c2915'),
    bike_name: 'Shine',
    model: 'gear',
    category: '100cc',
    colors_available: [ 'red', 'black' ],
    manufacturer: 'Honda',
    performance: 7,
    timestamp: '2020',
    price: 80000
  }

ehicles> db.four_wheelers.find()

  {
    _id: ObjectId('69b176024ebccd0ecf7c2916'),
    vehicle_name: 'Swift',
    model: 'own',
    category: 'car',
    variants: [ 'vxi', 'petrol' ],
    manufacturer: 'Maruti',
    performance: 8,
    timestamp: '2021',
    price: 700000
  },
  {
    _id: ObjectId('69b176024ebccd0ecf7c2917'),
    vehicle_name: 'Innova',
    model: 'own',
    category: 'car',
    variants: [ 'diesel' ],
    manufacturer: 'Toyota',
    performance: 9,
    timestamp: '2023',
    price: 2000000
  },
  {
    _id: ObjectId('69b176024ebccd0ecf7c2918'),
    vehicle_name: 'Bolero',
    model: 'own',
    category: 'car',
    variants: [ 'diesel' ],
    manufacturer: 'Mahindra',
    performance: 8,
    timestamp: '2022',
    price: 1000000
  },
```

```
{
  _id: ObjectId('69b176024ebccd0ecf7c2919'),
  vehicle_name: 'TataBus',
  model: 'commercial',
  category: 'bus',
  variants: [ 'diesel' ],
  manufacturer: 'Tata',
  performance: 7,
  timestamp: '2020',
  price: 4000000
},
{
  _id: ObjectId('69b176024ebccd0ecf7c291a'),
  vehicle_name: 'LeylandTruck',
  model: 'commercial',
  category: 'heavy truck',
  variants: [ 'diesel' ],
  manufacturer: 'Ashok Leyland',
  performance: 8,
  timestamp: '2022',
  price: 3500000
}
```

7. Write a MongoDB query to display only vehicle name and price in all the collection of the database.

Command:

db.two_wheelers.find({}, {bike_name:1,price:1,_id:0})

db.four_wheelers.find({}, {vehicle_name:1,price:1,_id:0})

```
vehicles> db.two_wheelers.find({}, {bike_name:1,price:1,_id:0})
[
  { bike_name: 'Activa', price: 90000 },
  { bike_name: 'Pulsar', price: 120000 },
  { bike_name: 'Apache', price: 150000 },
  { bike_name: 'FZ', price: 130000 },
  { bike_name: 'Shine', price: 80000 }
]
vehicles> db.four_wheelers.find({}, {vehicle_name:1,price:1,_id:0})
[
  { vehicle_name: 'Swift', price: 700000 },
  { vehicle_name: 'Innova', price: 2000000 },
  { vehicle_name: 'Bolero', price: 1000000 },
  { vehicle_name: 'TataBus', price: 4000000 },
  { vehicle_name: 'LeylandTruck', price: 3500000 }
]
```

8. Write a MongoDB query to display two_wheelers from a particular company.

Command:

db.two_wheelers.find({manufacturer:"Honda"})

```
vehicles> db.two_wheelers.find({manufacturer:"Honda"})
[
  {
    _id: ObjectId('69b175f04ebccd0ecf7c2911'),
    bike_name: 'Activa',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'red', 'black' ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: '2022',
    price: 90000
  },
  {
    _id: ObjectId('69b175f04ebccd0ecf7c2915'),
    bike_name: 'Shine',
    model: 'gear',
    category: '100cc',
    colors_available: [ 'red', 'black' ],
    manufacturer: 'Honda',
    performance: 7,
    timestamp: '2020',
    price: 80000
  }
]
```

9. Write a MongoDB query to display four_wheelers available in diesel variants

Command:

db.four_wheelers.find({variants:"diesel"})

```
vehicles> db.four_wheelers.find({variants:"diesel"})
[
  {
    _id: ObjectId('69b176024ebccd0ecf7c2917'),
    vehicle_name: 'Innova',
    model: 'own',
    category: 'car',
    variants: [ 'diesel' ],
    manufacturer: 'Toyota',
    performance: 9,
    timestamp: '2023',
    price: 2000000
  },
  {
    _id: ObjectId('69b176024ebccd0ecf7c2918'),
    vehicle_name: 'Bolero',
    model: 'own',
    category: 'car',
    variants: [ 'diesel' ],
    manufacturer: 'Mahindra',
    performance: 8,
    timestamp: '2022',
    price: 1000000
  },
  {
    _id: ObjectId('69b176024ebccd0ecf7c2919'),
    vehicle_name: 'TataBus',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel' ],
    manufacturer: 'Tata',
    performance: 7,
    timestamp: '2020',
    price: 4000000
  },
  {
    _id: ObjectId('69b176024ebccd0ecf7c291a'),
    vehicle_name: 'LeylandTruck',
    model: 'commercial',
    category: 'heavy truck',
    variants: [ 'diesel' ],
    manufacturer: 'Ashok Leyland',
    performance: 8,
    timestamp: '2022',
    price: 3500000
  }
]
```

10. Write a MongoDB query to display vehicles name, category and manufacturer details whose rating is more than 5.

Command:

db.two_wheelers.find({performance:{$gt:5}},
{bike_name:1,category:1,manufacturer:1,_id:0})

db.four_wheelers.find({performance:{$gt:5}},
{vehicle_name:1,category:1,manufacturer:1,_id:0})

```
vehicles> db.two_wheelers.find({performance:{$gt:5}},{bike_name:1,category:1,manufacturmanufacturer:1,_id:0})
[
  { bike_name: 'Activa', category: '125cc', manufacturer: 'Honda' },
  { bike_name: 'Pulsar', category: '150cc', manufacturer: 'Bajaj' },
  { bike_name: 'Apache', category: '200cc', manufacturer: 'TVS' },
  { bike_name: 'FZ', category: '150cc', manufacturer: 'Yamaha' },
  { bike_name: 'Shine', category: '100cc', manufacturer: 'Honda' }
]
vehicles> db.four_wheelers.find({performance:{$gt:5}},{vehicle_name:1,category:1,manufamanufacturer:1,_id:0})
[
  { vehicle_name: 'Swift', category: 'car', manufacturer: 'Maruti' },
  { vehicle_name: 'Innova', category: 'car', manufacturer: 'Toyota' },
  { vehicle_name: 'Bolero', category: 'car', manufacturer: 'Mahindra' },
  { vehicle_name: 'TataBus', category: 'bus', manufacturer: 'Tata' },
  {
    vehicle_name: 'LeylandTruck',
    category: 'heavy truck',
    manufacturer: 'Ashok Leyland'
  }
]
```

QUESTION 2. Use MongoDB to implement the following DB operations for a Zoo

1. Create a database called ‘animal’ and write a MongoDB query to select database as ‘animal’.

Command:

use animal

```
test> use animal
switched to db animal
```

2. Write a MongoDB query to display all the databases.

Command:

show dbs

```
animal> show dbs
admin      40.00 KiB
config     36.00 KiB
local      40.00 KiB
vehicles   80.00 KiB
```

3. Create a collection called ‘wild_animals’.(use capping) and Create a collection called ‘domestic_animals’.

Command:

db.createCollection("wild_animals",{capped:true,size:50000})

db.createCollection("domestic_animals")

```
animal> db.createCollection("wild_animals",{capped:true,size:50000})
{ ok: 1 }
animal> db.createCollection("domestic_animals")
{ ok: 1 }
```

4. Add 5 wild_animal details to the collection named ‘wild_animals’. Each document consists of following fields as animal_name, nature (harm or harmless), favorite_foods (meat, rabbits, deer etc) as array, care_taker_name, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

Command:

```
db.wild_animals.insertMany([
{
animal_name:"Lion",
nature:"harm",
favorite_foods:["meat","deer"],
care_taker_name:"Ravi",
life_span:15,
timestamp:"2022",
expenses:50000
},
{
animal_name:"Tiger",
nature:"harm",
favorite_foods:["meat","rabbit"],
care_taker_name:"Arun",
life_span:18,
timestamp:"2021",
expenses:60000
},
{
animal_name:"Elephant",
nature:"harmless",
favorite_foods:["grass","fruits"],
care_taker_name:"Suresh",
life_span:60,
timestamp:"2020",
expenses:80000
},
{
animal_name:"Bear",
nature:"harm",
favorite_foods:["fish","meat"],
care_taker_name:"Ravi",
life_span:20,
```

timestamp:"2023",
expenses:45000
},
{
animal_name:"Deer",
nature:"harmless",
favorite_foods:["grass","plants"],
care_taker_name:"Kumar",
life_span:12,
timestamp:"2022",
expenses:20000
}
])

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b1958f1ca54c64087c2907'),
    '1': ObjectId('69b1958f1ca54c64087c2908'),
    '2': ObjectId('69b1958f1ca54c64087c2909'),
    '3': ObjectId('69b1958f1ca54c64087c290a'),
    '4': ObjectId('69b1958f1ca54c64087c290b')
  }
}
```

5. Add 5 domestic-animal details to the collection named ‘domestic_animals’. Each document consists of following fields as animal_name, gender (male or female), favorite_foods (meat, rabbits, deer etc) as array, animal_petname, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

Command:

db.domestic_animals.insertMany([
{
animal_name:"Dog",
gender:"male",
favorite_foods:["meat","biscuits"],
animal_petname:"Tommy",
life_span:12,
timestamp:"2023",
expenses:10000
},
{
animal_name:"Cat",
gender:"female",
favorite_foods:["fish","milk"],

animal_petname:"Kitty",
life_span:15,
timestamp:"2022",
expenses:8000
},
{
animal_name:"Cow",
gender:"female",
favorite_foods:["grass","fodder"],
animal_petname:"Gauri",
life_span:20,
timestamp:"2021",
expenses:15000
},
{
animal_name:"Goat",
gender:"male",
favorite_foods:["plants","leaves"],
animal_petname:"Rocky",
life_span:10,
timestamp:"2020",
expenses:7000
},
{
animal_name:"Rabbit",
gender:"female",
favorite_foods:["carrot","grass"],
animal_petname:"Bunny",
life_span:8,
timestamp:"2022",
expenses:5000
}
])

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b195a01ca54c64087c290c'),
    '1': ObjectId('69b195a01ca54c64087c290d'),
    '2': ObjectId('69b195a01ca54c64087c290e'),
    '3': ObjectId('69b195a01ca54c64087c290f'),
    '4': ObjectId('69b195a01ca54c64087c2910')
  }
}
```

6. Write a MongoDB query to display all documents available in wild_animals anddomastic_animals.

Command:

db.wild_animals.find()

db.domestic_animals.find()

```
[
  {
    _id: ObjectId('69b1958f1ca54c64087c2907'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ravi',
    life_span: 15,
    timestamp: '2022',
    expenses: 50000
  },
  {
    _id: ObjectId('69b1958f1ca54c64087c2908'),
    animal_name: 'Tiger',
    nature: 'harm',
    favorite_foods: [ 'meat', 'rabbit' ],
    care_taker_name: 'Arun',
    life_span: 18,
    timestamp: '2021',
    expenses: 60000
  },
  {
    _id: ObjectId('69b1958f1ca54c64087c2909'),
    animal_name: 'Elephant',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'fruits' ],
    care_taker_name: 'Suresh',
    life_span: 60,
    timestamp: '2020',
    expenses: 80000
  },
  {
    _id: ObjectId('69b1958f1ca54c64087c290a'),
    animal_name: 'Bear',
    nature: 'harm',
    favorite_foods: [ 'fish', 'meat' ],
    care_taker_name: 'Ravi',
    life_span: 20,
    timestamp: '2023',
    expenses: 45000
  },
  {
    _id: ObjectId('69b1958f1ca54c64087c290b'),
    animal_name: 'Deer',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'plants' ],
    care_taker_name: 'Kumar',
    life_span: 12,
    timestamp: '2022',
    expenses: 20000
  }
]
```

```
[
  {
    _id: ObjectId('69b195a01ca54c64087c290c'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [ 'meat', 'biscuits' ],
    animal_petname: 'Tommy',
    life_span: 12,
    timestamp: '2023',
    expenses: 10000
  },
  {
    _id: ObjectId('69b195a01ca54c64087c290d'),
    animal_name: 'Cat',
    gender: 'female',
    favorite_foods: [ 'fish', 'milk' ],
    animal_petname: 'Kitty',
    life_span: 15,
    timestamp: '2022',
    expenses: 8000
  },
  {
    _id: ObjectId('69b195a01ca54c64087c290e'),
    animal_name: 'Cow',
    gender: 'female',
    favorite_foods: [ 'grass', 'fodder' ],
    animal_petname: 'Gauri',
    life_span: 20,
    timestamp: '2021',
    expenses: 15000
  },
  {
    _id: ObjectId('69b195a01ca54c64087c290f'),
    animal_name: 'Goat',
    gender: 'male',
    favorite_foods: [ 'plants', 'leaves' ],
    animal_petname: 'Rocky',
    life_span: 10,
    timestamp: '2020',
    expenses: 7000
  },
  {
    _id: ObjectId('69b195a01ca54c64087c2910'),
    animal_name: 'Rabbit',
    gender: 'female',
    favorite_foods: [ 'carrot', 'grass' ],
    animal_petname: 'Bunny',
    life_span: 8,
    timestamp: '2022',
    expenses: 5000
  }
]
```

7. Write a MongoDB query to display only animal name and expenses in all the collection of the database.

Command:

db.wild_animals.find({}, {animal_name:1,expenses:1,_id:0})

db.domestic_animals.find({}, {animal_name:1,expenses:1,_id:0})

```
animal> db.wild_animals.find({}, {animal_name:1,expenses:1,_id:0})
[
  { animal_name: 'Lion', expenses: 50000 },
  { animal_name: 'Tiger', expenses: 60000 },
  { animal_name: 'Elephant', expenses: 80000 },
  { animal_name: 'Bear', expenses: 45000 },
  { animal_name: 'Deer', expenses: 20000 }
]
animal> db.domestic_animals.find({}, {animal_name:1,expenses:1,_id:0})
[
  { animal_name: 'Dog', expenses: 10000 },
  { animal_name: 'Cat', expenses: 8000 },
  { animal_name: 'Cow', expenses: 15000 },
  { animal_name: 'Goat', expenses: 7000 },
  { animal_name: 'Rabbit', expenses: 5000 }
]
```

8. Write a MongoDB query to display domestic_animals whose life is a particular year.

Command:

db.domestic_animals.find({life_span:10})

```
animal> db.domestic_animals.find({life_span:10})
[
  {
    _id: ObjectId('69b195a01ca54c64087c290f'),
    animal_name: 'Goat',
    gender: 'male',
    favorite_foods: [ 'plants', 'leaves' ],
    animal_petname: 'Rocky',
    life_span: 10,
    timestamp: '2020',
    expenses: 7000
  }
]
```

9. Write a MongoDB query to display wild_animals available under a particular care_taker.

Command:

db.wild_animals.find({care_taker_name:"Ravi"})

```
animal> db.wild_animals.find({care_taker_name:"Ravi"})
[
  {
    _id: ObjectId('69b1958f1ca54c64087c2907'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ravi',
    life_span: 15,
    timestamp: '2022',
    expenses: 50000
  },
  {
    _id: ObjectId('69b1958f1ca54c64087c290a'),
    animal_name: 'Bear',
    nature: 'harm',
    favorite_foods: [ 'fish', 'meat' ],
    care_taker_name: 'Ravi',
    life_span: 20,
    timestamp: '2023',
    expenses: 45000
  }
]
```

10. Write a MongoDB query to display animal name, favorite_foods and expenses details whose lifespan is more than 5 years.

Command:

db.wild_animals.find({life_span:{$gt:5}}, {animal_name:1,favorite_foods:1,expenses:1,_id:0})

db.domestic_animals.find({life_span:{$gt:5}}, {animal_name:1,favorite_foods:1,expenses:1,_id:0})

```
animal> db.wild_animals.find({life_span:{$gt:5}}, {animal_name:1,favorite_foods:1,expenexpenses:1,_id:0})
[
  {
    animal_name: 'Lion',
    favorite_foods: [ 'meat', 'deer' ],
    expenses: 50000
  },
  {
    animal_name: 'Tiger',
    favorite_foods: [ 'meat', 'rabbit' ],
    expenses: 60000
  },
  {
    animal_name: 'Elephant',
    favorite_foods: [ 'grass', 'fruits' ],
    expenses: 80000
  },
  {
    animal_name: 'Bear',
    favorite_foods: [ 'fish', 'meat' ],
    expenses: 45000
  },
  {
    animal_name: 'Deer',
    favorite_foods: [ 'grass', 'plants' ],
    expenses: 20000
  }
]
```

```
animal> db.domestic_animals.find({life_span:{$gt:5}}, {animal_name:1,favorite_foods:1,eexpenses:1,_id:0})
[
  {
    animal_name: 'Dog',
    favorite_foods: [ 'meat', 'biscuits' ],
    expenses: 10000
  },
  {
    animal_name: 'Cat',
    favorite_foods: [ 'fish', 'milk' ],
    expenses: 8000
  },
  {
    animal_name: 'Cow',
    favorite_foods: [ 'grass', 'fodder' ],
    expenses: 15000
  },
  {
    animal_name: 'Goat',
    favorite_foods: [ 'plants', 'leaves' ],
    expenses: 7000
  },
  {
    animal_name: 'Rabbit',
    favorite_foods: [ 'carrot', 'grass' ],
    expenses: 5000
  }
]
```